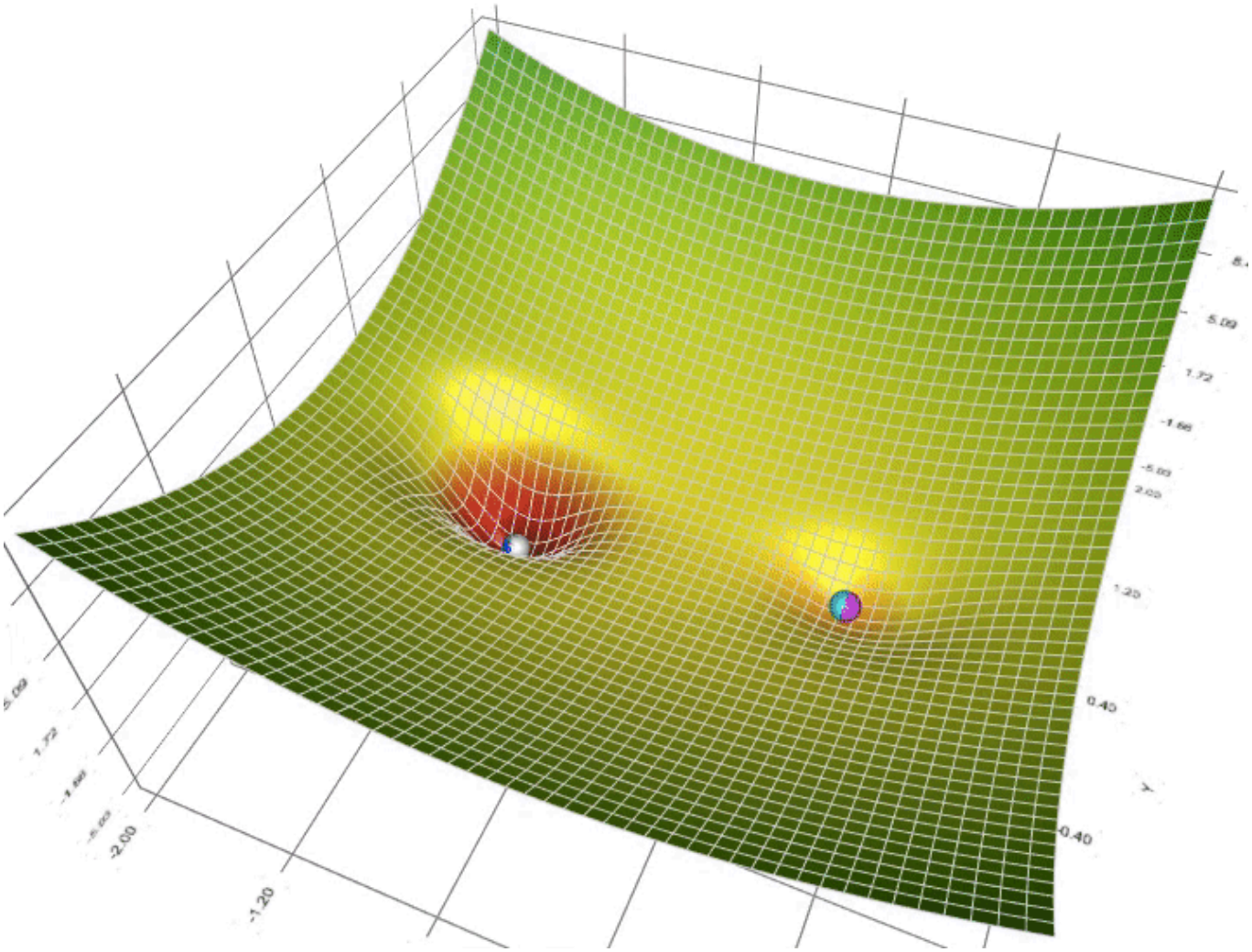


Animation of 5 gradient descent methods on a surface: gradient descent (cyan), momentum (magenta), AdaGrad (white), RMSProp (green), Adam (blue). Left well is the global minimum; right well is a local minimum.

A Visual Explanation of Gradient Descent Methods (Momentum, AdaGrad, RMSProp, Adam)

Lili Jiang · Follow

Published in Towards Data Science · 9 min read · Jun 7, 2020

With a myriad of resources out there explaining gradient descents, in this post, I'd like to visually walk you through how each of these methods works. With the aid of a gradient descent visualization tool I built, hopefully I can present you with some unique insights, or minimally, many GIFs.

I assume basic familiarity of why and how gradient descent is used in machine learning (if not, I recommend this video by 3Blue1Brown) . My focus here is to compare and contrast these methods. If you are already familiar with all the methods, you can scroll to the bottom to watch a few fun "horse races".

Vanilla Gradient Descent

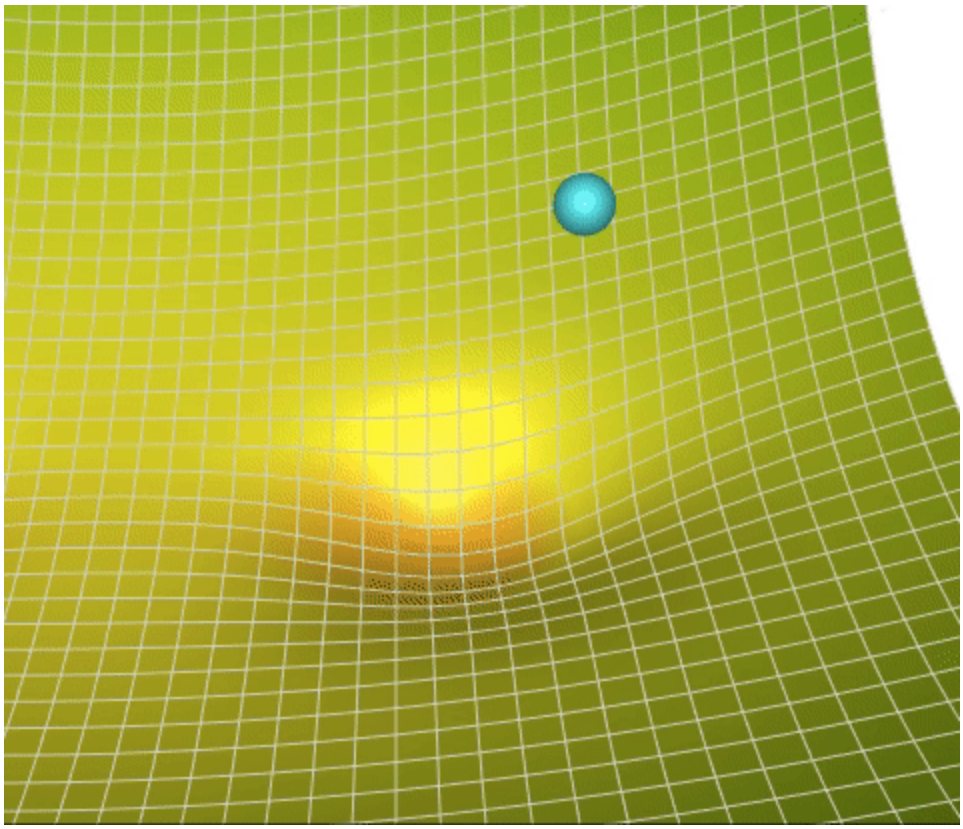
Let's have a quick refresher. In the context of machine learning, the goal of gradient descent is usually to minimize the loss function for a machine learning problem. A good algorithm finds the minimum fast and reliably well (i.e. it doesn't get stuck in local minima, saddle points, or plateau regions, but rather goes for the global minimum).

The basic gradient descent algorithm follows the idea that the opposite direction of the gradient points to where the lower area is. So it iteratively takes steps in the opposite directions of the gradients. For each parameter theta, it does the following:

*delta = - learning_rate * gradient*

theta += delta

Theta is some parameter you want to optimize (for example, the weight of a neuron-to-neuron connection in neural network, the coefficient for a feature in linear regression, etc). There can be thousands of such *thetas* in an ML optimization setting. *Delta* is the amount of change for *theta* after every iteration in the algorithm; the hope is that with each of such change, *theta* is incrementally getting closer to the optimum.



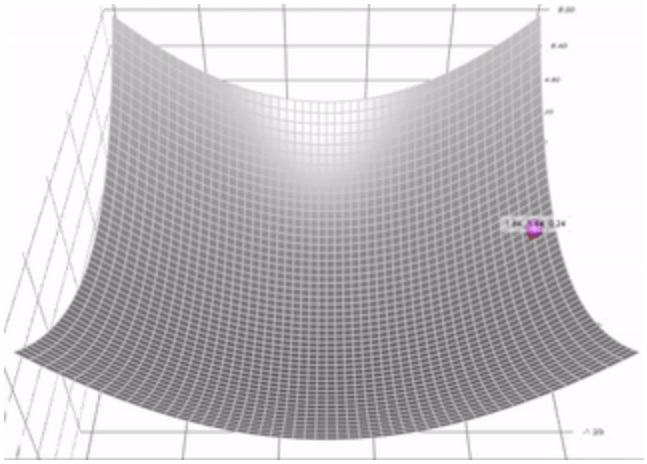
Step-by-step illustration of gradient descent algorithm.

As human perception is limited to 3-dimensions, in all my visualizations, imagine we only have two parameters (or *thetas*) to optimize, and they are represented by the x and y dimension in the graph. The surface is the loss function. We want to find the (x, y) combination that's at the lowest point of the surface. The problem is trivial to us because we can see the whole surface. But the ball (the descent algorithm) doesn't; it can only take one step

at a time and explore its surroundings, analogous to walking in the dark with only a flashlight.

The vanilla gradient descent is vanilla because it just operates on the gradients. The following methods do some additional processing of the gradients to be faster and better.

Momentum



Momentum descent with decay_rate = 1.0 (no decay).

The gradient descent with momentum algorithm (or Momentum for short) borrows the idea from physics. Imagine rolling down a ball inside of a frictionless bowl. Instead of stopping at the bottom, the momentum it has accumulated pushes it forward, and the ball keeps rolling back and forth.

We can apply the concept of momentum to our vanilla gradient descent algorithm. In each step, in addition to the regular gradient, it also adds on the movement from the previous step. Mathematically, it is commonly expressed as:

delta = - *learning_rate* * *gradient* + ***previous_delta * decay_rate*** (eq. 1)

theta += *delta* (eq. 2)

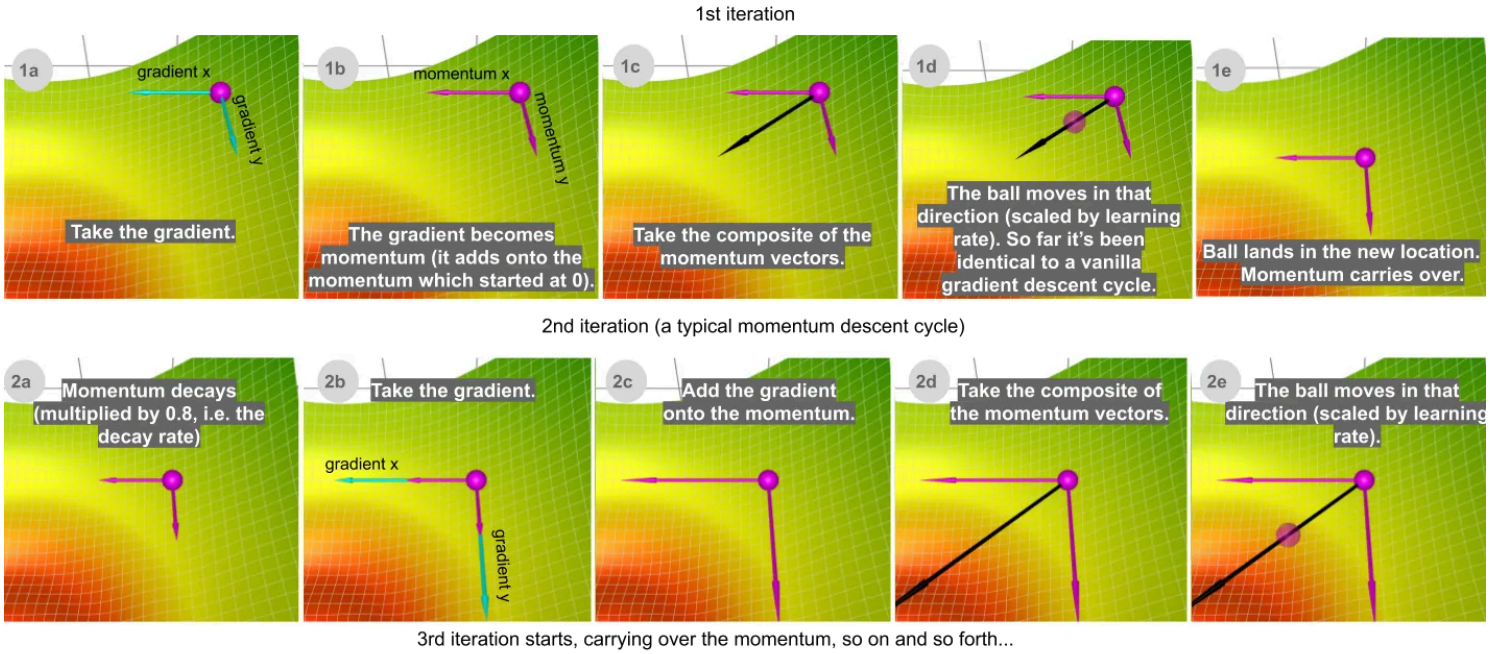
I found it more intuitive if I massage this equation a little and keep track of the (decayed) cumulative sum of gradient instead. This will also make things easier when we introduce the Adam algorithm later.

*sum_of_gradient = gradient + previous_sum_of_gradient * decay_rate* (eq. 3)

*delta = -learning_rate * sum_of_gradient* (eq. 4)

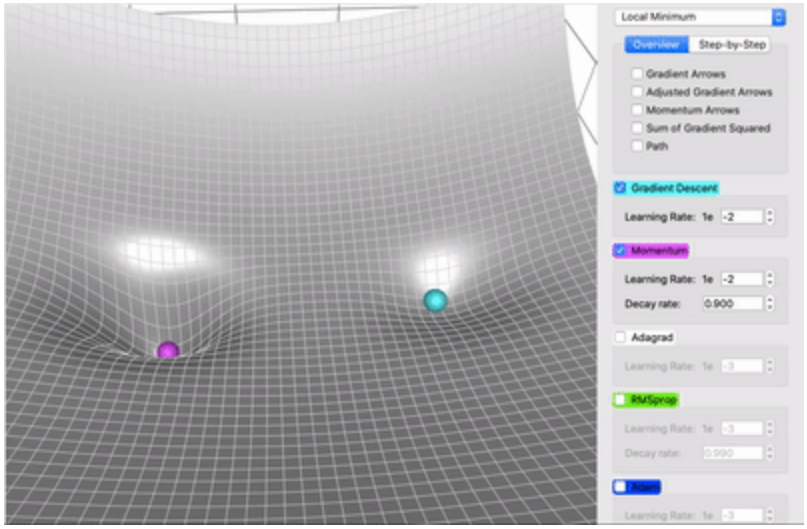
theta += *delta* (eq. 5)

(What I did was factoring out -learning_rate. To see the mathematical equivalence, you can substitute delta with -learning_rate * sum_of_gradient in eq. 1 to get eq. 3.)



Step-by-step illustration of momentum descent. Watch live animation in the app. For the rest of this post, I sloppily use gradient x and gradient y in the visualization; in reality, because it's gradient *descent*, it's actually the negative of the gradient.

Let's consider two extreme cases to understand this decay rate parameter better. If the decay rate is 0, then it is exactly the same as (vanilla) gradient descent. If the decay rate is 1 (and provided that the learning rate is reasonably small), then it rocks back and forth endlessly like the frictionless bowl analogy we mentioned in the beginning; you do not want that. Typically the decay rate is chosen around 0.8–0.9 — it's like a surface with a little bit of friction so it eventually slows down and stops.



Momentum (magenta) vs. Gradient Descent (cyan) on a surface with a global minimum (the left well) and local minimum (the right well)

So, in what ways is Momentum better than vanilla gradient descent? In this comparison on the left, you can see two advantages:

1. Momentum simply moves faster (because of all the momentum it accumulates)
2. Momentum has a shot at escaping local minima (because the momentum may propel it out of a local minimum). In a similar vein, as we shall see later, it will also power through plateau regions better.

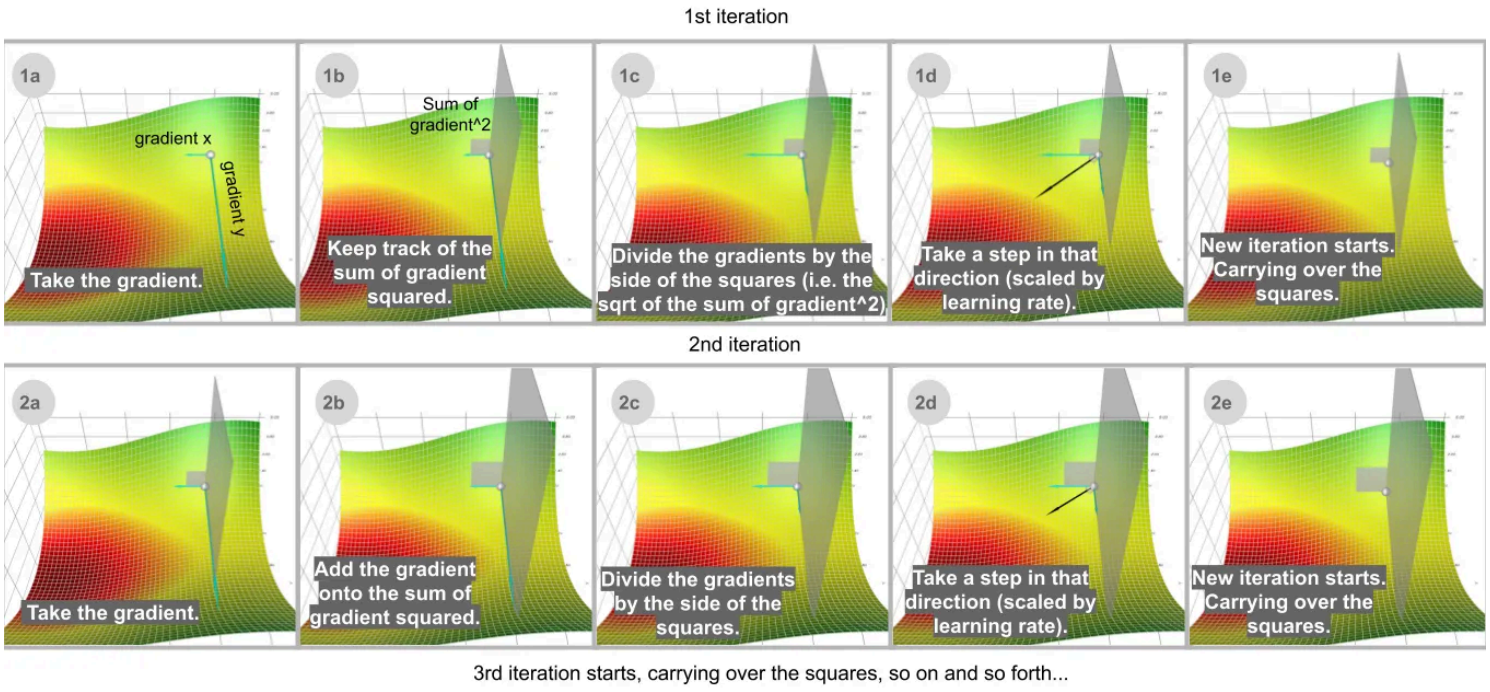
AdaGrad

Instead of keeping track of the sum of gradient like momentum, the **Ada**ptive **Grad**ient algorithm, or AdaGrad for short, keeps track of the sum of gradient *squared* and uses that to adapt the gradient in different directions. Often the equations are expressed in tensors. I will avoid tensors to simplify the language here. For each dimension:

$$sum\_of\_gradient\_squared = previous\_sum\_of\_gradient\_squared + gradient^2$$

$$delta = -learning\_rate * gradient \,/\, \mathbf{sqrt(sum\_of\_gradient\_squared)}$$

$$theta \mathrel{+}= delta$$

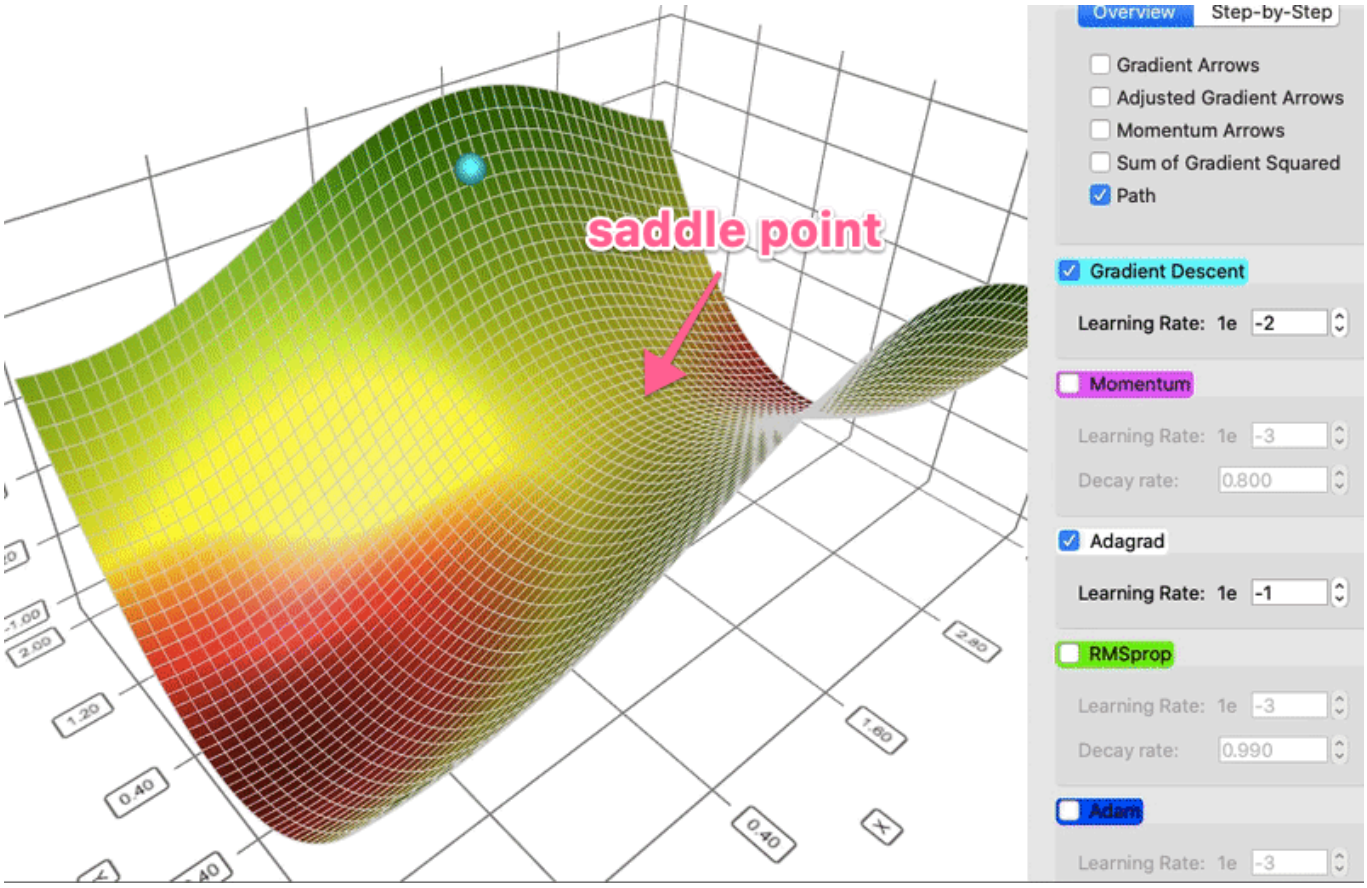


Step-by-step illustration of AdaGrad descent. Watch live animation in the app.

In ML optimization, some features are very sparse. The average gradient for sparse features is usually small so such features get trained at a much slower

rate. One way to address this is to set different learning rates for each feature, but this gets messy fast.

AdaGrad addresses this problem using this idea: the more you have updated a feature already, the less you will update it in the future, thus giving a chance for the others features (for example, the sparse features) to catch up. In visual terms, how much you have updated this feature is to say how much you have moved in this dimension, and this notion is captured by the cumulative sum of gradient squared. Notice how in the step-by-step grid illustration above, without the rescaling adjustment (1b), the ball would have moved mostly vertically downwards; with the adjustment (1d), it moves diagonally.



AdaGrad (white) vs. gradient descent (cyan) on a terrain with a saddle point. The learning rate of AdaGrad is set to be higher than that of gradient descent, but the point that AdaGrad's path is straighter stays largely true regardless of learning rate.

This property allows AdaGrad (and other similar gradient-squared-based methods like RMSProp and Adam) to escape a saddle point much better. AdaGrad will take a straight path, whereas gradient descent (or relatedly, Momentum) takes the approach of "let me slide down the steep slope first and *maybe* worry about the slower direction later". Sometimes, vanilla gradient descent might just stop at the saddle point where gradients in both directions are 0 and be perfectly content there.

RMSProp

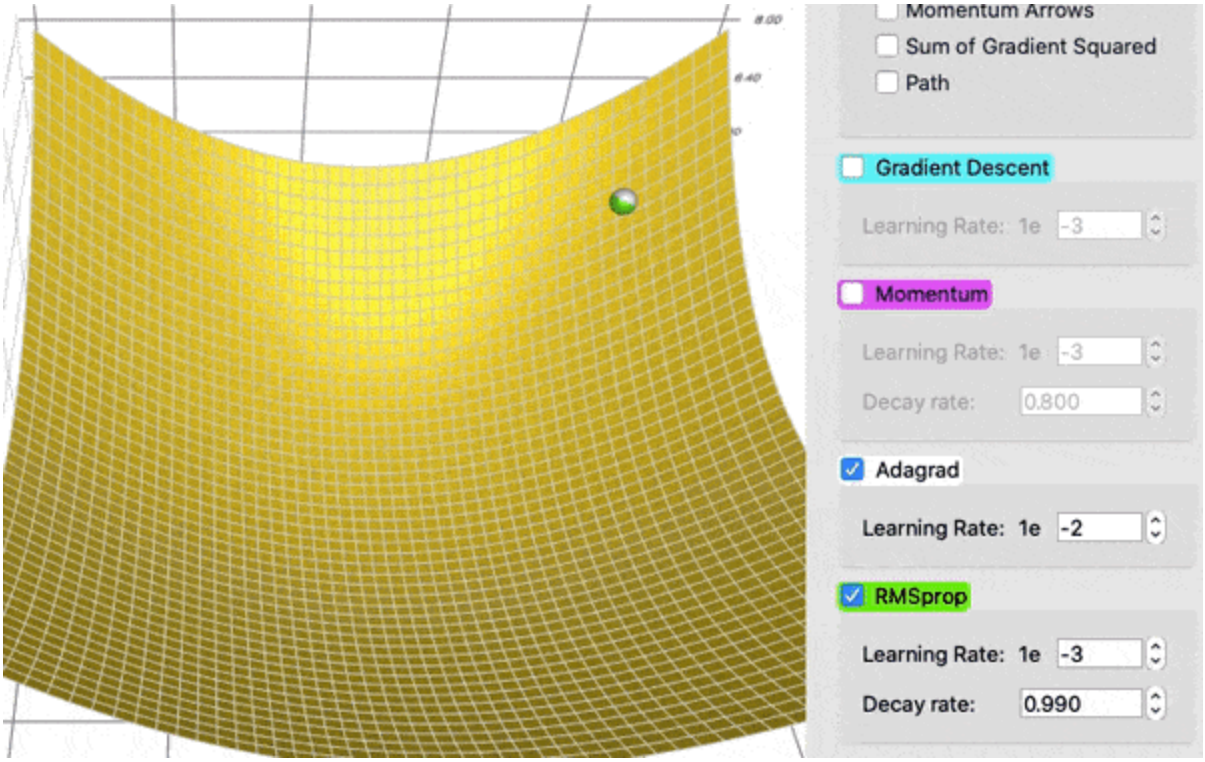
The problem of AdaGrad, however, is that it is incredibly slow. This is because the *sum of gradient squared* only grows and never shrinks. RMSProp (for **R**oot **M**ean **S**quare **Prop**agation) fixes this issue by adding a decay factor.

sum_of_gradient_squared = previous_sum_of_gradient_squared **** decay_rate*** *+ gradient*^{2} **** (1- decay_rate)***

*delta = -learning_rate * gradient / sqrt(sum_of_gradient_squared)*

theta += delta

More precisely, the sum of gradient squared is actually the *decayed* sum of gradient squared. The decay rate is saying only recent gradient^2 matters, and the ones from long ago are basically forgotten. As a side note, the term "decay rate" is a bit of a misnomer. Unlike the decay rate we saw in momentum, in addition to decaying, the decay rate here also has a scaling effect: it scales down the whole term by a factor of (1 - decay_rate). In other words, if the decay_rate is set at 0.99, in addition to decaying, the sum of gradient squared will be sqrt(1 - 0.99) = 0.1 that of AdaGrad, and thus the step is on the order of 10x larger for the same learning rate.



RMSProp (green) vs AdaGrad (white). The first run just shows the balls; the second run also shows the sum of gradient squared represented by the squares.

To see the effect of the decaying, in this head-to-head comparison, AdaGrad white) keeps up with RMSProp (green) initially, as expected with the tuned learning rate and decay rate. But the *sums of gradient squared* for AdaGrad accumulate so fast that they soon become humongous (demonstrated by the sizes of the squares in the animation). They take a heavy toll and eventually AdaGrad practically stops moving. RMSProp, on the other hand, has kept the squares under a manageable size the whole time, thanks to the decay rate. This makes RMSProp faster than AdaGrad.

Adam

Last but not least, Adam (short for **Ada**ptive **M**oment Estimation) takes the best of both worlds of Momentum and RMSProp. Adam empirically works

Let's take a look at how it works:

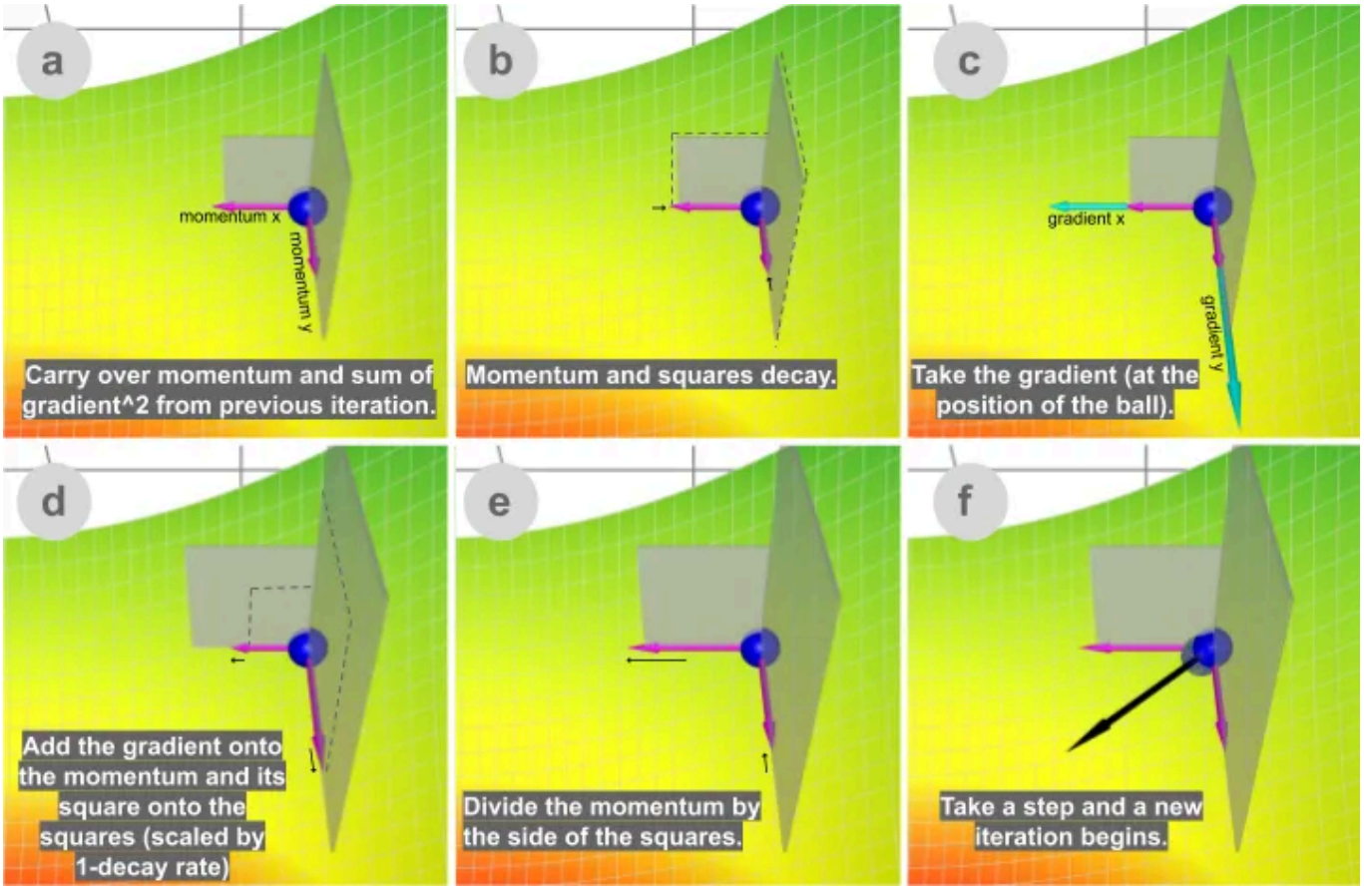
*sum_of_gradient = previous_sum_of_gradient * beta1 + gradient * (1 - beta1)* [Momentum]

*sum_of_gradient_squared = previous_sum_of_gradient_squared * beta2 + gradient² * (1- beta2)* [RMSProp]

*delta = -learning_rate * sum_of_gradient / sqrt(sum_of_gradient_squared)*

theta += delta

Beta1 is the decay rate for the first moment, sum of gradient (aka momentum), commonly set at 0.9. Beta 2 is the decay rate for the second moment, sum of gradient squared, and it is commonly set at 0.999.

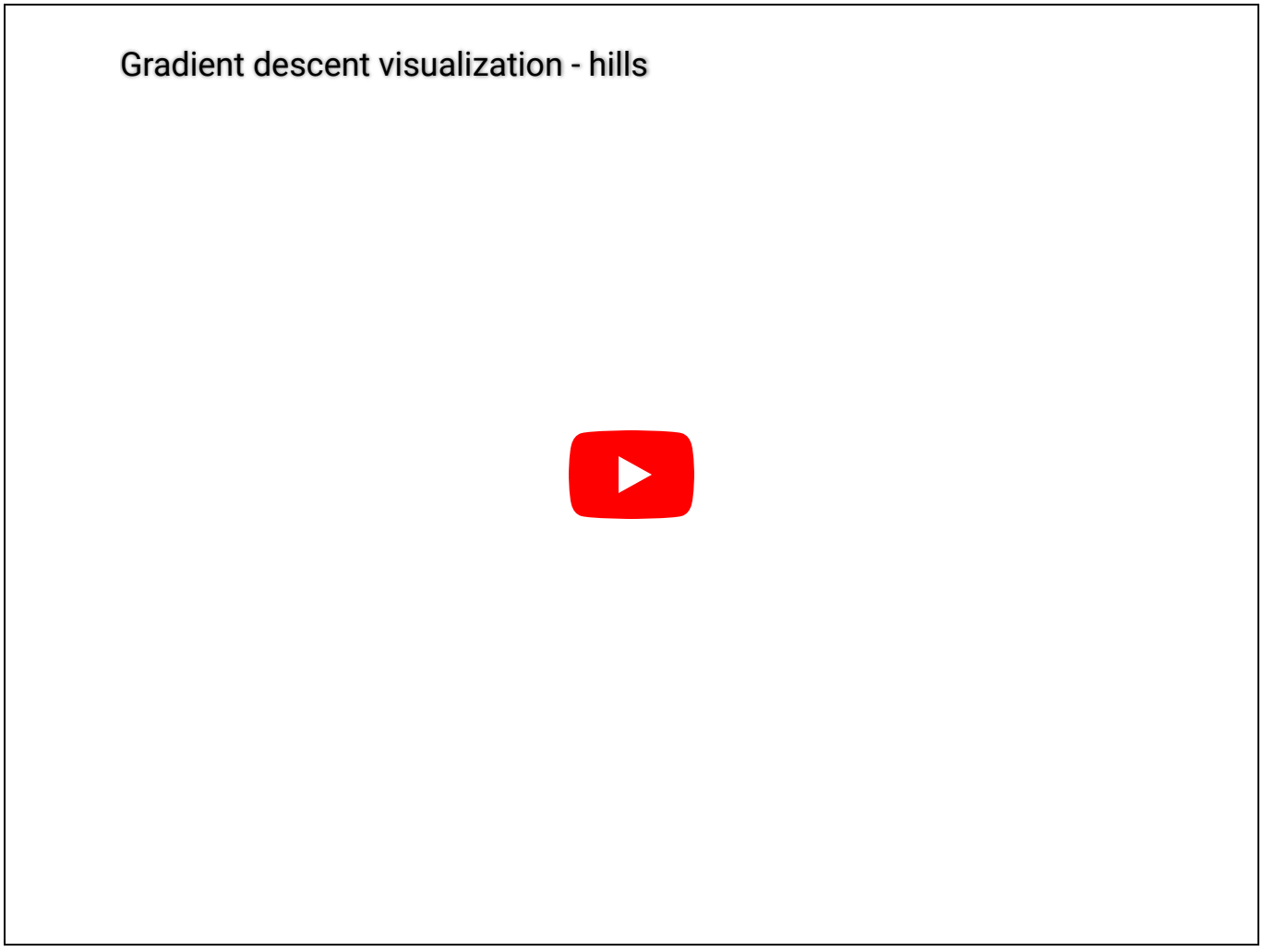


Step-by-step illustration of Adam descent. Watch live animation in the app.

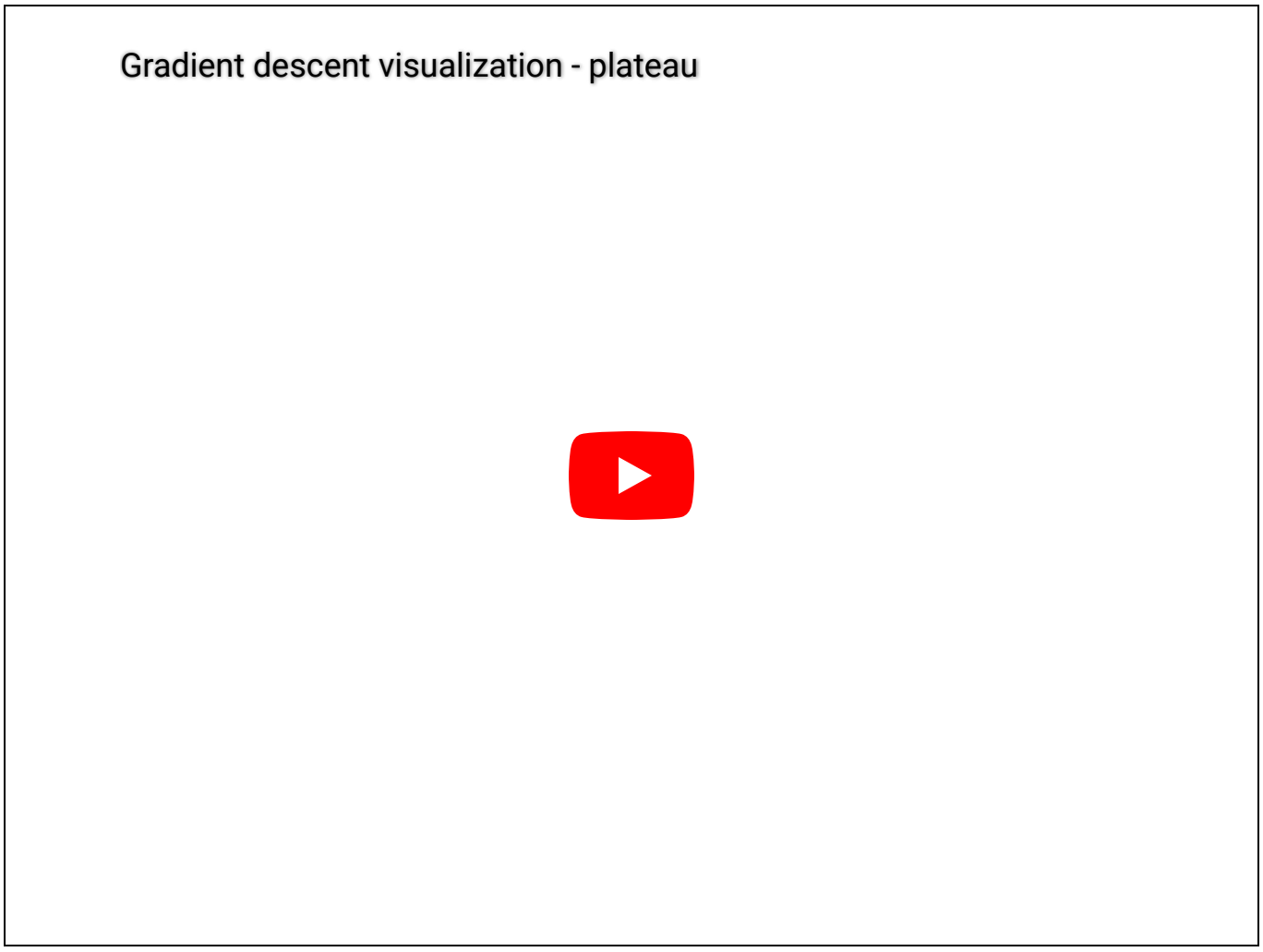
Adam gets the speed from momentum and the ability to adapt gradients in different directions from RMSProp. The combination of the two makes it powerful.

Closing Words

Now that we have discussed all the methods, let's watch a few races of all the descent methods we talked about so far! (There is some inevitable cherry-picking of parameters. The best way to get a taste is to play around yourself.)



In this terrain, there are two little hills blocking the way to the global minimum. Adam is the only one able to find its way to the global minimum. Whichever way the parameters are tuned, from this starting position at least, none of the other methods can get there. This means neither momentum nor adaptive gradient alone can do the trick. It's really the combination of the two: first, momentum takes Adam beyond the local minimum where all the other balls stop; then, the adjustment from the sum of gradient squared pulls it sideway, because it is the direction less explored, leading to its final victory.



Here is another race. In this terrain, there is a flat region (plateau) surrounding the global minimum. With some parameter tuning, Momentum

and Adam (thanks to its momentum component) can make it to the center, while the other methods can't.

In summary, gradient descent is a class of algorithms that aims to find the minimum point on a function by following the gradient. Vanilla gradient descent just follows the gradient (scaled by learning rate). Two common tools to improve gradient descent are the sum of gradient (first moment) and the sum of the gradient squared (second moment). The Momentum method uses the first moment with a decay rate to gain speed. AdaGrad uses the second moment with no decay to deal with sparse features. RMSProp uses the second moment by with a decay rate to speed up from AdaGrad. Adam uses both first and second moments, and is generally the best choice. There are a few other variations of gradient descent algorithms, such as Nesterov accelerated gradient, AdaDelta, etc., that are not covered in this post.

Lastly I shall leave you with this momentum descent with no decay. Its path makes up a fun pattern. I see no practical use (yet) but present it here just for the funsies. [Edit: I take my word back about no practical use. Read more about this curve at https://en.wikipedia.org/wiki/Lissajous_curve.]

Play around the visualization tool that's used to generate all the visualization in this post, and see what you find!

References and relevant links:

[1] http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

[2] https://ruder.io/optimizing-gradient-descent

[3] https://bl.ocks.org/EmilienDupont/aaf429be5705b219aaaf8d691e27ca87

Machine Learning | Gradient Descent | Data Visualization | Data Science

Written by Lili Jiang

Follow

755 Followers · Writer for Towards Data Science

Eng @ Waymo; Fmr. Head of Data Science @ Quora

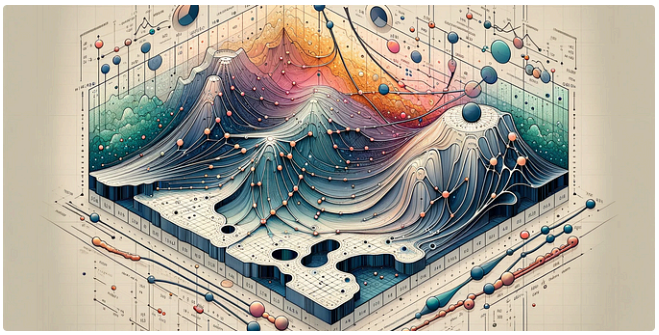
More from Lili Jiang and Towards Data Science

Lili Jiang in Towards Data Science

How GPT works: A Metaphoric Explanation of Key, Value, Query i...

GPT, explained simply, in a metaphor of potion.

10 min read · Jun 17, 2023



Cristian Leo in Towards Data Science

The Math behind Adam Optimizer

Why is Adam the most popular optimizer in Deep Learning? Let's understand it by diving...

16 min read · Jan 30, 2024

641 8

Siavash Yasini in Towards Data Science

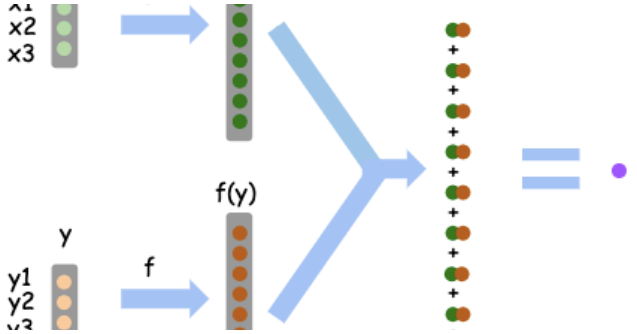
Python's Most Powerful Decorator

And 5 ways to use it in data science and machine learning

✨ · 11 min read · Feb 2, 2024

2.4K 17

2.3K 19



Lili Jiang in Towards Data Science

An Intuitive Explanation of Kernels in Support Vector Machine (SVM)

We will walk through a simple example with basic arithmetics to demystify the concept o...

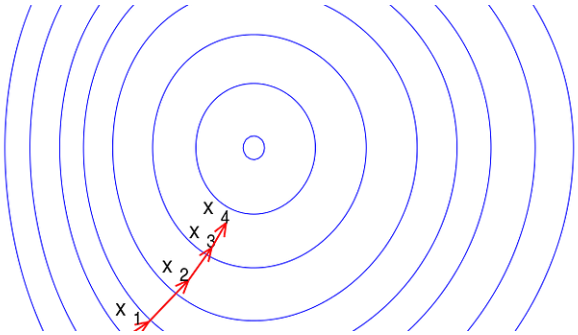
4 min read · Apr 5, 2020

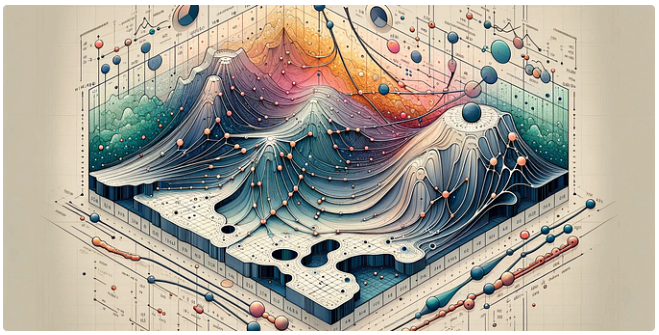
143

See all from Lili Jiang

See all from Towards Data Science

Recommended from Medium





Kartik Chaudhary in Game of Bits

Understanding Optimizers for training Deep Learning Models

Learn about popular SGD variants knows as optimizers

8 min read · Nov 15, 2023

112

Cristian Leo in Towards Data Science

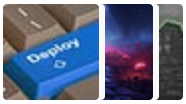
The Math behind Adam Optimizer

Why is Adam the most popular optimizer in Deep Learning? Let's understand it by diving...

16 min read · Jan 30, 2024

2.3K 19

Lists



Predictive Modeling w/ Python

20 stories · 967 saves

Practical Guides to Machine Learning

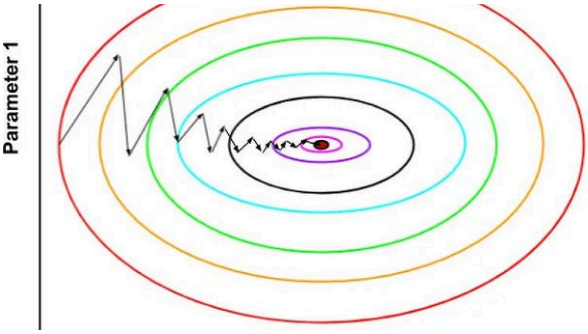
10 stories · 1147 saves

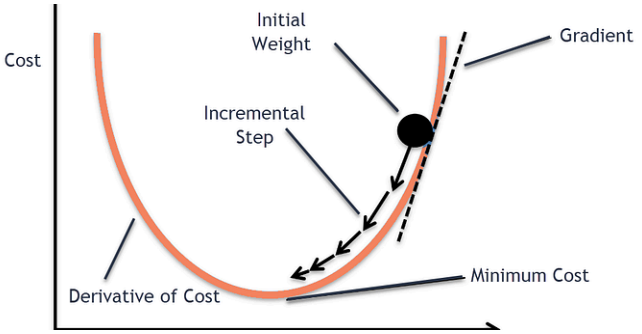
Natural Language Processing

1253 stories · 734 saves

The New Chatbots: ChatGPT, Bard, and Beyond

12 stories · 322 saves






Francesco Franco

ADAM optimization in machine learning

In two previous posts we went over three of the most powerful and popular optimization...

7 min read · Oct 29, 2023

86


Soham Shirsat

Gradient descent

Introduction

5 min read · Nov 27, 2023

668 1

Juan C Olamendy

Understanding ReLU, LeakyReLU, and PReLU: A Comprehensive...

Why should you care about ReLU and its variants in neural networks?

6 min read · Dec 4, 2023

36 1

Benedict Neo in bitgrit Data Science Publication

Roadmap to Learn AI in 2024

A free curriculum for hackers and programmers to learn AI

11 min read · Feb 21, 2024

3.9K 44

See more recommendations